

Lab Sheet 8: Software Testing & CI/CD

Objectives

By completing today's workshop, you will:

- Write functional **Unit Tests** for your backend application logic.
- Implement **Integration Tests** to verify your API routes and Dockerized service communication.
- Automate your testing strategy by building a **Continuous Integration (CI)** pipeline using GitHub Actions.

Introduction: Securing your Codebase

In Assessment 1, you are building a critical system (a Robot Ground Control Station). You must ensure that users cannot bypass Role-Based Access Control (RBAC) and that invalid coordinates are not sent to the physical (or virtual) robot. Manual testing is too slow. Today, you will establish an automated testing safety net.

*Note: The examples below use Python and **pytest**. If you are building a Node.js backend, you should adapt these concepts using **Jest** or **Mocha**.*

Task 1: Writing Unit Tests (The Base of the Pyramid)

Goal: Test small, isolated pieces of your application logic.

The Concept: Unit tests do not need a database or network connection. They test raw logic. Let's test a hypothetical coordinate validator.

Action:

1. In your backend codebase, create a new folder called `tests/`.
2. Create a file called `test_validation.py`.
3. Write a simple function and an accompanying test. Notice the "Arrange, Act, Assert" pattern:

```
# The logic (usually imported from your main code)
def is_valid_target(x, y):
    if x < 0 or x > 100 or y < 0 or y > 100:
        return False
    return True

# The test
def test_valid_coordinates():
    # Arrange (Setup inputs)
    good_x, good_y = 50, 50
```

```
bad_x, bad_y = 150, -10

# Act (Run the function)
result_good = is_valid_target(good_x, good_y)
result_bad = is_valid_target(bad_x, bad_y)

# Assert (Check the outcomes)
assert result_good == True
assert result_bad == False
```

4. Run your test locally in the terminal (e.g., by typing `pytest` or `npm test`). Ensure it passes.

Task 2: API Integration Testing

Goal: Test that your web server responds correctly to HTTP requests.

The Concept: Integration tests ensure that the routing and request handling work. For instance, if someone tries to hit your `/api/move` endpoint without an authentication token, it should reject them.

Action:

1. In your `tests/` folder, create `test_api.py`.
2. Write an integration test that hits your local application framework. (If using FastAPI/Flask in Python, you can use a `TestClient`. If using Express in Node, use `Supertest`).

```
from fastapi.testclient import TestClient
from main import app # Import your actual app

client = TestClient(app)

def test_unauthorized_move():
    # Attempting to move without a token
    response = client.post("/api/move", json={"x": 10, "y": 10})

    # The API should block this and return a 401 Unauthorized status
    assert response.status_code == 401
```

3. **Extend the Test:** Write a second test that passes a valid "Viewer" role token. Assert that the API returns a 403 Forbidden (because Viewers cannot move the robot).

Task 3: Automating with GitHub Actions (CI/CD)

Goal: Ensure your tests run automatically every time you push code to GitHub.

The Concept: You have written tests, but humans forget to run them. Continuous Integration (CI) uses a server to spin up an environment, install your dependencies, and run the tests automatically on every commit.

Action:

1. In the root directory of your repository, create the following folder structure: `.github/workflows/`
2. Inside that folder, create a file called `ci-tests.yml`.
3. Paste in the following YAML configuration. (Modify the setup steps if you are using Node.js instead of Python):

```
name: Automated Testing Pipeline

# Trigger this workflow on every push to the main branch
on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Check out repository code
        uses: actions/checkout@v3

      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: "3.10"

      - name: Install dependencies
        run: |
          pip install pytest fastapi
          # Add any other requirements.txt installations here

      - name: Run test suite
        run: |
          pytest
```

4. Commit and push this file to GitHub.
5. Go to your GitHub repository in the browser, click the **Actions** tab, and watch your pipeline build and test your code automatically!

Task 4: Test Coverage Reporting

Goal: Generate a report detailing exactly what percentage of your codebase is covered by tests.

The Concept: Code coverage tools analyze your application while the tests run to see which lines of code were executed and which were ignored.

Action:

1. Install a coverage tool (e.g., `pytest-cov` for Python or `nyc` for Node.js).
2. Run your tests with the coverage flag: `pytest --cov=.`

3. Review the terminal output. It will tell you the percentage of code covered. Your goal as an engineer is generally to maintain $> 80\%$ coverage for critical systems.
4. *Bonus*: Add this coverage step to your GitHub Actions YAML file so the coverage report is generated automatically on the server!

Further Resources

- **PyTest Documentation:** docs.pytest.org - *The standard testing framework for Python.*
- **Jest Documentation:** jestjs.io - *A delightful JavaScript Testing Framework with a focus on simplicity.*
- **GitHub Actions Quickstart:** docs.github.com/en/actions - *Official guide to setting up automated workflows.*